

Operating Systems (CSE2430)

Lab Session 5: Security

March 22, 2018

Deadline: 21 March 2025 - 12:30

Introduction

Session 5 of this lab allows you to explore system security issues. You are asked to exploit a simple application to execute a sensitive function and in the second part you will write a secure loader that only allows signed programs to be executed.

3.1 Designing a Stack Exploit

Familiarize yourself with stack exploits, e.g., by reading *PicoCTF 2022: Buffer Overflow Series* <https://enscribe.dev/blog/picoctf-2022/buffer-overflow#intro>. You will learn about different techniques that can be used to design an exploit for a vulnerable program.

Write a stack exploit for the following program, which is also included in the attachments:

```
#include <stdio.h>

void vulnerable() __attribute__((section (".secret")));

void vulnerable() {
    printf("I have been exploited!\n");
}

int main() {
    setbuf(stdout, 0);
    setbuf(stdin, 0);
    char buf[32];
    printf("Please tell me your student number: ");
    gets(buf);
    printf("Your student number is %s\n", buf);
}
```

Analyze why exactly the above program is vulnerable, then create and implement a strategy for an exploit. A simple solution requires disabling stack protection when compiling the program with

GCC by passing. You can do this by using the provided `comp.sh` file. You do not need to understand what the `linker.ld` and `comp.sh` files do. You also do not need to understand what the first line after the includes does.

In case you cannot build the program, a pre-built binary for x64 Linux is also available on Brightspace. More advanced (return-oriented programming) approaches might not need it. The expected outcome of your exploit is that without any modifications to the program and just by applying your exploit the hidden function is invoked.

Deliverables You are expected to hand in:

1. A program/shell script/python3 script which prints out your payload, so you can pipe it into the binary.

You are expected to be able to demonstrate your exploit and explain your payload.

3.2 Writing a Secure Loader

In this exercise you are asked to write a secure program loader that only executes programs which have been signed and rejects programs with missing or wrong signature.

Asymmetric cryptography works using keypairs of a public and a private key. The private key can be used to sign data. The corresponding public key can be used to verify signatures. Furthermore, the public key can be used to encrypt data which can then only be decrypted using the corresponding private key. The private key should not be shared with third parties, while the public key should.

Figure 1 shows the transition between plaintext and encrypted data. Encryption uses the public key (making it possible for any third party to encrypt data). Decryption uses the private key (which only the owner of the key has, therefore only they can decrypt the data).

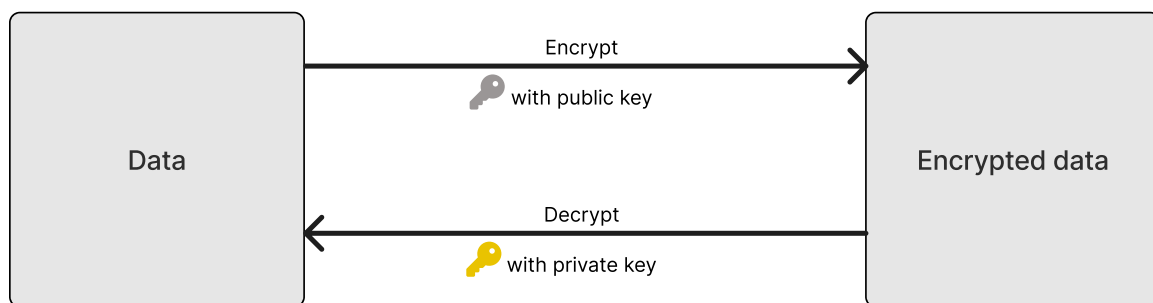


Figure 1: Encrypting and decrypting data.

Figure 2 shows the transition between plaintext and signed data. Signing is done using the private key (so only the owner can sign data). Unsigning (retrieving the original from the signed data) is done using the public key. In practice, the data that is signed and unsigned is a hash.

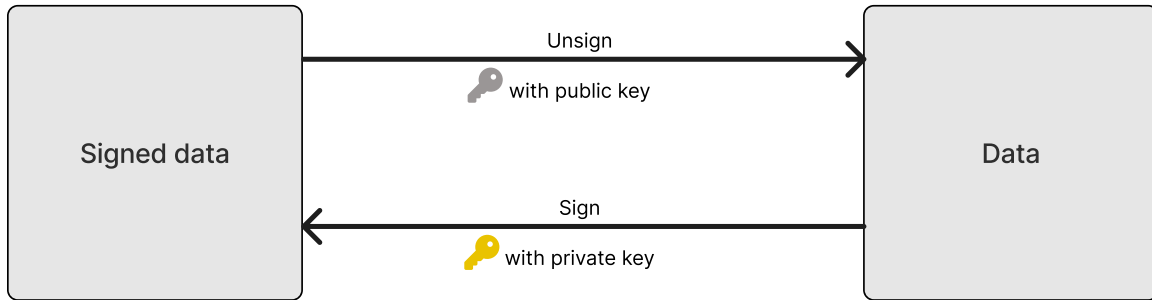


Figure 2: Signing data and retrieving the original.

Do you see similarities between the two figures above? Fun fact: the process of signing data is mathematically equivalent to decryption, and the process of unsigning data is equivalent to encryption. In the figures, the public key can only make a transition to the right, while the private key can make a transition to the left. To sign some data, you essentially "decrypt" it (even though it is not encrypted). After that, anyone with the public key can "encrypt" this "decrypted" data and get the original (plaintext) data. If this process results in data which makes sense, then they can be sure that this data was first "decrypted" (signed) with the private key.

The idea behind secured programs is to create a secure digest (hash) using `openssl` and to embed the hash into the binary as an ELF section named `.sha`. Your loader can now, with the knowledge of the public key, verify the integrity of the program binary before executing it. Binaries that do not contain a valid secure digest or no digest at all should be rejected by the loader.

Figure 3 shows visually how the process of signing a binary and verifying its signature should work. Your task is to implement only the second half (signature verification).

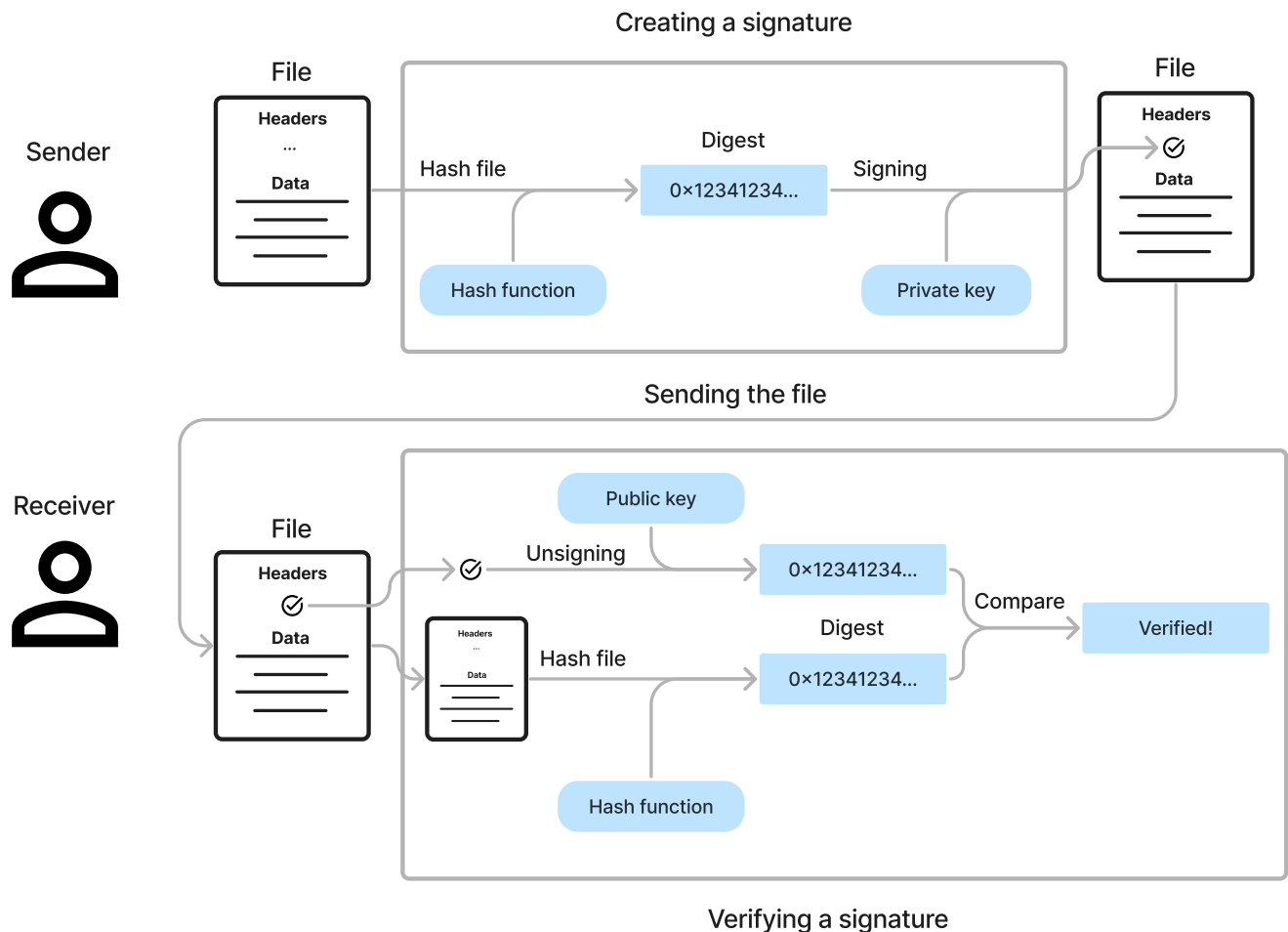


Figure 3: Signing and verifying a file.

Please note that adding a header to the ELF binary is a modification and therefore **the signature header must be removed again before verification**. If the header is not removed, the hash of the binary will be different than the original (before the insertion of the header), therefore verification will always fail.

In order to implement the loader, you first have to create an RSA public/private key pair. In the first step, you create the private key:

```
openssl genpkey -algorithm RSA -out private_key.pem
```

This key needs to remain a secret. Set the file system access permissions so that only you can read it:

```
chmod 400 private_key.pem
```

Now you can extract the public key from the private key:

```
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

Useful tools for this exercise are `openssl dgst` and `objcopy`.

You can find an example secured binary, unsecured and secured with a different key and a public key for testing on Brightspace.

Deliverables You are expected to hand in:

1. A loader written as a sh/bash/python3 script which accepts the provided usage pattern. It's important that the loader executes correctly signed binaries and rejects everything else.

```
loader <secured binary> <public key>
```